

Software Development Life Cycle (SDLC) Selection for the Project.

Group-03

Name	ID
Afsana Hena	2252421109
Marjia Khatun	23524202071
Mihir Bormon	23524202121
Alvee Ahnaf Khan	23524202141

Steps-01:Analyzing the problem

a. System Requirements:

- User registration as Restaurant or Orphanage with email verification
- Secure login using email and password
- Restaurants can post food donations with details like Food Item Name, Quantity, Pickup Deadline, Special Instructions etc.
- Donations saved with status "Available"
- Orphanages can view list of available donations
- Sort donations by Pickup Deadline or Date Posted
- Display contact details to both parties upon claiming
- Restaurants can mark claimed donations as "Picked Up"
- History log for completed donations
- Support 50+ simultaneous users
- Passwords hashed and salted, data encrypted
- Protection against SQL Injection
- Crash recovery and auto-restart
- Responsive UI across all devices
- Liability waiver and food safety disclaimer

b. Project scope:

In Scope:

- Registration for restaurants and orphanages
- Posting food donations
- Searching and listing available donations
- Claiming items

- Notifications/messaging for pickup coordination
- History logs for transparency

Out of Scope:

- Physical delivery of food
- Payment processing
- Monitoring food temperature in transit

c. Understand user needs:

User Class	User Needs
Restaurant Partner	Fast, minimal-typing interface to alert nearby orphanages about leftover food
Orphanage Partner	One-stop dashboard with reliable info on what's offered, quantity, and pickup windows
System Administrator	Account approvals, platform monitoring, dispute resolution, impact reports

Step-02: Evaluate Project Characteristics

a. Requirement Stability: Stable

The SRS document for the Food Waste Management System is well-defined and complete. All functional requirements such as registration, posting donations, claiming items, notifications, history logs etc, and non-functional requirements such as security, performance, responsiveness, crash recovery etc are clearly specified. While minor UI refinements may arise during development based on feedback from restaurant staff and orphanage managers, the core functionality is unlikely to change. Therefore, the requirements are considered stable.

b. Risk Level: Medium

The Food Waste Management System has several identifiable risks, but they are manageable. There are some key risks like system downtime leading to wasted food, data privacy breaches exposing phone numbers and addresses, food safety incidents from expired or mishandled items, and impersonation risks like fake restaurants or orphanages. These risks can be mitigated through proper authentication, encryption, liability waivers, and platform monitoring. Since the system

does not handle payments, physical delivery, or life-critical operations, the overall risk level is medium.

c. Complexity: Medium

This is a Java-based web application with a MySQL relational database, HTML/CSS/JavaScript frontend, and real-time notifications. It supports three user roles: Restaurant, Orphanage, System Administrator with different permissions. The system requires responsive design for desktops, tablets, and phones. However, there is no integration with third-party delivery services, payment gateways, or IoT devices. The architecture is straightforward, making the overall complexity medium.

d. Project Delivery Timeline: Moderate

The project aims to reduce food waste and fight hunger, so a reasonably timely delivery is important. However, it is not an emergency or safety-critical system that requires immediate deployment. A working Minimum Viable Product (MVP) with core features can be delivered in 2 to 3 months. Additional features like advanced sorting, detailed history logs, impact reports etc can follow in subsequent releases. The timeline is considered moderate.

Step-03: Compare SDLC Models

Table: Comparison matrix with different models

Criteria	Priority	Waterfall	V-Shape	Iterative	Spiral	Agile	Prototype
5	Well-defined requirements	Yes	Yes	No	Yes	Yes	No
5	System reliability & security	No	Yes	No	Yes	Yes	No
4	User feedback & UI importance	No	No	Yes	Yes	Yes	Yes
5	Risk management (food safety, impersonation)	No	Yes	No	Yes	Yes	No
3	Structured testing at each stage	No	Yes	No	Yes	Yes	No

3	Early delivery of core functionality	No	No	Yes	No	Yes	Yes
3	Flexibility for requirement changes	No	No	Yes	Yes	Yes	Yes
2	Customer involvement during development	No	No	Yes	Yes	Yes	Yes
Total Score - 30	Over all	5	18	12	27	30	12

Step 4: Select Suitable Model: Agile

Step 5: Justify Selection

The results are as follows: Waterfall (5), V-Shaped (18), Iterative (12), Spiral (27), Agile(30), and Prototype (12). The Agile model is the most suitable choice for the Food Waste Management System because it achieved a perfect score by meeting every project criterion, including well-defined requirements, security, UI importance, risk management, structured testing, early delivery, flexibility, and customer involvement. The SRS explicitly requires an intuitive interface where users can post or claim food within given time on any device, and Agile's iterative sprints with continuous stakeholder feedback allow rapid UI refinements based on real input from busy restaurant staff and orphanage managers. Risk management for food safety, impersonation, and data privacy is handled through sprint-based security testing and regular code reviews, while early delivery of a Minimum Viable Product (MVP) with core features (registration, posting, claiming) can start reducing food waste immediately, with advanced features like notifications and history logs added later. Flexibility is also critical, as real-world feedback may reveal needs like "quick-listing templates" for restaurants during rush hours, which Agile accommodates without derailing the project. In contrast, Spiral (27/30) adds heavy risk analysis overhead suited for large, high-risk systems like aerospace or medical devices, making it overkill for this medium web app. V-Shaped (18/30) lacks flexibility for UI improvements and early delivery. Prototype (12/30) would add unnecessary throwaway development time since requirements are already clear. Iterative (12/30) lacks structured testing and risk management critical for food safety and data privacy, and Waterfall (5/30) is too rigid, delaying security fixes and UI issues until final launch. Therefore, Agile is the optimal model for this project.